

## COMP0008: Computer Architecture & Concurrency – CW3

Dear students,

The questions below will not be marked, and they are for you to test your understanding.

We will publish solutions in 1-2 weeks that will allow you to check your answers.

If any of the questions are not clear, please post a question in the forum or attend the drop-in sessions.

COMP0008 staff.

### Part 1: Scrutinizing MIPS's ISA:

In the lecture, we discussed the difference between `addi` and `addiu`.

- a. Write a short MIPS assembly program that throws an exception when executing the following instruction:  

`addi $t0, $t0, -1`
- b. Change the above to `addiu` and check that no exception is thrown. Can you give an example of a case where such an instruction is useful?

### Part 2: Theoretical Questions:

Which of the following claims are correct?

- ❖ It is better to write an assembly program that uses as few registers as possible (even if it means performing more instructions for the same task).
- ❖ Say that programs A and B each execute exactly 1000 instructions to perform a task, but A only uses `$t0` while B modifies many more registers. Then, A will be faster than B due to caching.
- ❖ In a function, the callee is responsible for restoring the values of `$a0-$a3` before the function ends.
- ❖ If a function does not call another function, it is more efficient to avoid storing `$ra` on the stack.
- ❖ If a function does not call another function, we should not use the memory for efficiency (as we don't have to modify any of the stored registers).
- ❖ Using `jal label` is essentially the same as doing (1) `la $t0, label` and (2) `jalr $t0`.

### Part 3: Recursion in MIPS:

In the lecture, we discussed additive (e.g., the Roman) and positional (e.g., decimal) number systems.

In some older additive systems (e.g., [the Egyptian](#)), there was no importance to the order of symbols.

Here, we will consider the Weird0008 number system. It is an additive number system that uses the digits 1-9 (i.e., without zero).

The Weird0008 system requires that digits will be sorted in non-increasing order, and thus allows multiple representations of the same numbers.

Your task is to write a MIPS program that takes as input a number  $n > 0$  and outputs all its Weird0008 representations.

For example, if  $n = 5$ , the output can be:

5

41

32

311

221

2111

11111

When programming, try to follow the register usage convention (use  $\$t0$ - $\$t9$  for temporary variables,  $\$s0$ - $\$s7$  for saved variables,  $\$a0$ - $\$a3$  for arguments, etc.).